**Default Value of the Parameter**

The default value is false.

**Return Value**

The return value has the data type *any*.

It designates the created graphic, compare the method *_3D.getGraphic*.

**Examples**

```
var g := Station._3D.getGraphic("default").importGraphics("C:\Program
Files\Siemens\Plant Simulation 2606\3D\jt-graphics\Robot Kuka.jt", true,
true)
g.Position.Z := 3
g.Name := "RobotKuka"
```

```
myFrame._3D.getGraphic("deco").importGraphics("D:\\MyDWGFolder\
\myLayout.dwg")
```

**See also**

**Import Graphics [command]**

**_3D.getGraphic [SimTalk]**

**optimizeByPruningTinyGraphics [SimTalk]**

**optimizeByStructureFlattening [SimTalk]**

**optimizeByVisibilityFilter [SimTalk]**

# Accessing Attributes of Graphic Shapes

## _Accessing Attributes of Graphic Shapes

*SimTalk* provides the *attributes*, *read-only attributes*, and *methods* listed in the table of contents to the left for setting properties of **shapes**.

**See also**

**Insert Shape**

**Accessing Properties of Graphic Shapes**

## Dimensions [SimTalk] - barred area

Sets the dimensions of the *graphic of the barred area* designated by <Path>.

### Remarks

The attribute is available for graphics of type **BarredArea**, compare *InternalGraphicType*.

### Type

Attribute

### Syntax

```
<Path>.Dimensions:length[2]
```

### Assignment Value

You can assign an *array* of data type *length* with two values.

They designate the **Dimension X** and **Dimension Y** of the barred area.

### Example

```
MyFrame._3D.getGraphic("deco", [2]).Dimensions := [4, 5]
```

### SimTalk

**InternalGraphicType [SimTalk]**

**createBarredArea [SimTalk]**

**X [SimTalk] - array**

**Y [SimTalk] - array**

**_3D.getGraphic [SimTalk]**

### See also

**Barred Area**

**Barred Area Settings**

## Dimensions [SimTalk] - graphic

Sets the dimensions of the *graphic* designated by <Path>.

### Remarks

The attribute is available for graphics of these types: **Box**, **FactoryWalls**, **Fence**, **Mezzanine**, and **Rack**, compare *InternalGraphicType*.

### Type

Attribute

### Syntax

```
<Path>.Dimensions:length[3]
```

### Assignment Value

You can assign an *array* of data type *length* with three values.

They designate the **Dimension X**, the **Dimension Y**, and the **Dimension Z** of the barred area.

> **Note**
> For a graphic of type **Rack** the attribute sets the entire size of the rack as such.

### Example

```
MyFrame._3D.getGraphic("deco", [2]).Dimensions := [4, 5, 2]
```

### SimTalk

InternalGraphicType [SimTalk]

### See also

| | |
|---|---|
| Box > Box Settings | createMezzanine [SimTalk] |
| createBox [SimTalk] | _Mezzanine Attributes |
| Factory Walls Factory Walls Settings | Rack > Rack Settings |
| createFactoryWalls [SimTalk] | createRack [SimTalk] |

| _Factory Walls Attributes | _Rack Attributes |
| --- | --- |
| Fence > Fence Settings | X [SimTalk] - array |
| createFence [SimTalk] | Y [SimTalk] - array |
| _Factory Walls Attributes | Z [SimTalk] - array |
| Mezzanine > Mezzanine Settings | _3D.getGraphic [SimTalk] |

## Form [SimTalk]

Sets the shape of the *barred area* designated by <Path>.

### Remarks

The attribute is available for graphics of type **BarredArea**, compare *InternalGraphicType*.

### Type

Attribute

### Syntax

```
<Path>.Form:string
```

### Assignment Value

You can assign a value of data type *string*.

You can specify `"Rectangular"`, `"Rectangular with hatching"`, `"Round"`, or `"One-sided delimitation"`.

### Examples

```
var ba := Station._3D.getGraphic("default").createBarredArea([4, 4])
// creates a rectangular barred arrea around the Station
```

```
var BarredArea := _3D.getGraphic("deco").createBarredArea([8, 6])
// the x dimension does not matter as long as it is not 0 as it
// will be discarded when the Form changes to "One-sided delimitation"
barredArea.Form := "One-sided delimitation"
```

### SimTalk

**InternalGraphicType [SimTalk]**

**createBarredArea [SimTalk]**

**_3D.getGraphic [SimTalk]**

### See also

**Barred Area**

## Barred Area Settings

## HasCutouts [SimTalk]

Returns if parts of the graphic of the factory walls, of the fence, or of the mezzanine designated by <Path> are cut out (true) or not (false).

### Remarks

The attribute is available for graphics of these types: **FactoryWalls**, **Fence**, and **Mezzanine**, compare *InternalGraphicType*.

### Type

Read-only attribute

### Syntax

```
<Path>.HasCutouts → boolean
```

### Return Value

The return value has the data type *boolean*.

### Example

```
print .Models.MyModel._3D.getGraphic("deco", [4]).HasCutouts
```

### SimTalk

**InternalGraphicType [SimTalk]**

### See also

| | |
|---|---|
| **Factory Walls** > Factory Walls Settings | **Mezzanine** > Mezzanine Settings |
| **createFactoryWalls [SimTalk]** | **createMezzanine [SimTalk]** |
| **_Factory Walls Attributes** | **_Mezzanine Attributes** |
| **Fence** > Fence Settings | **restoreCutouts [SimTalk]** |
| **createFence [SimTalk]** | **_3D.getGraphic [SimTalk]** |
| **_Factory Walls Attributes** | |

## PlaceRemainder [SimTalk]

Sets where the **remainder** of the elements of the factory walls or the fence designated by <Path> is to be added.

### Remarks

The remainder is the difference between the length of the wall/fence and the next lower multiple of the element width.

> **Note**
> The attribute is available for graphics of type **FactoryWalls** and **Fence**, compare *InternalGraphicType*.

### Type

Attribute

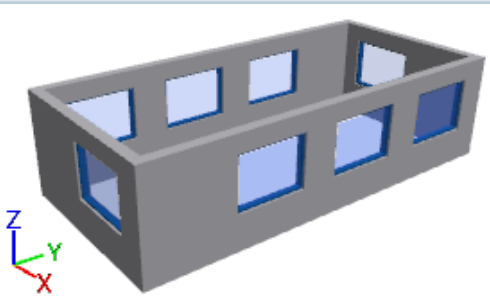
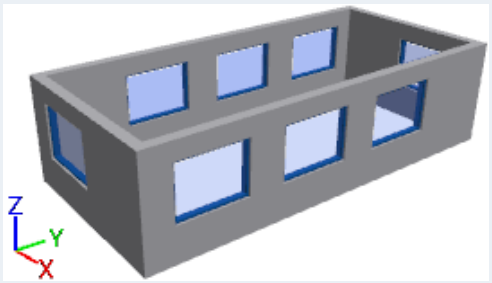
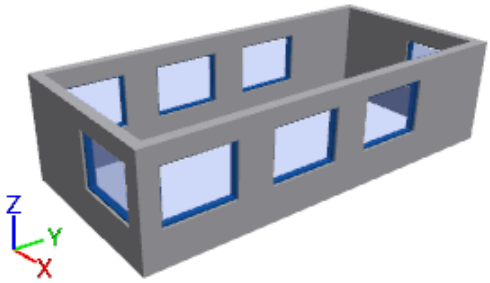
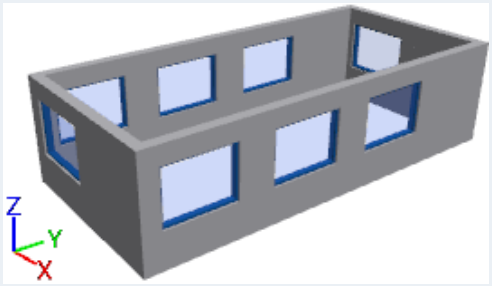
### Syntax

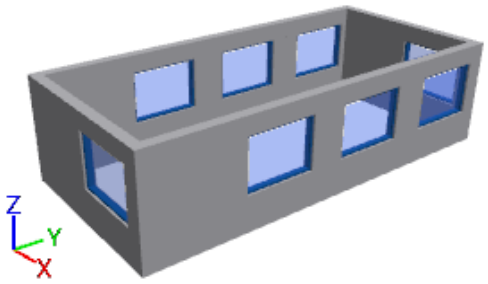
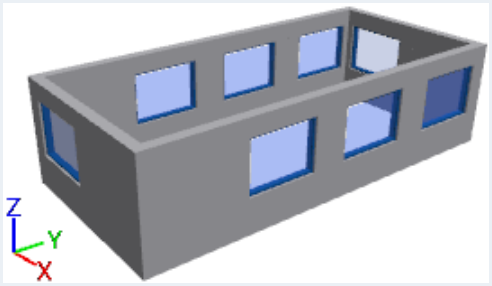
```
<Path>.PlaceRemainder:string
```

### Assignment Value

You can assign a value of data type *string*.

You can specify one of the following settings:

| Setting | Remainder is located | Looks like this |
|---|---|---|
| `"Clockwise"` | clockwise at the end of the wall |  |
| `"Counter-clockwise"` | counter-clockwise at the end of the wall |  |
| `"Top and left"` | at the top and the left end of the wall<br><br>depends on the orientation of the wall |  |
| `"Top and right"` | at the top and the right end of the wall<br><br>depends on the orientation of the wall |  |

| Setting | Remainder is located | Looks like this |
|---|---|---|
| `"Bottom and left"` | at the bottom and the left end of the wall<br><br>depends on the orientation of the wall |  |
| `"Bottom and right"` | at the bottom and the right end of the wall<br><br>depends on the orientation of the wall |  |

**Note**

When you change the setting for **Placing The Remainder**, *Plant Simulation* resets the **Cutouts**.

### Examples

```
MyStation._3D.getGraphic("default").createFactoryWalls([11,40,3],
42cm).PlaceRemainder := "Counter-clockwise"
```

```
MyStation1._3D.getGraphic("default").createFence([3,4,1], true,
false).PlaceRemainder := "Top and left"
```

### SimTalk

**InternalGraphicType [SimTalk]**

### See also

| | |
|---|---|
| **Factory Walls** > Factory Walls Settings | **createFence [SimTalk]** |
| **createFactoryWalls [SimTalk]** | **_Factory Walls Attributes** |
| **_Factory Walls Attributes** | **restoreCutouts [SimTalk]** |
| **Fence** > Fence Settings | **_3D.getGraphic [SimTalk]** |

## restoreCutouts [SimTalk]

Restores those parts of the graphic designated by <Path> that were cut out and returns them to the state before you cut out parts.

### Remarks

The method is available for graphics of these types: **FactoryWalls**, **Fence**, and **Mezzanine**, compare *InternalGraphicType*.

Assigning the method deactivates graphic inheritance.

You cannot assign the method if the graphic is an automatically generated graphic group or if the graphic is located in an automatically generated graphic group.

### Type

Method

### Syntax

```
<Path>.restoreCutouts -> boolean
```

### Return Value

The return value has the data type *boolean*.

### Example

```
print .Models.MyModel._3D.getGraphic("deco", [4]).restoreCutouts
```

### SimTalk

**InternalGraphicType [SimTalk]**

### See also

| | |
|---|---|
| **Factory Walls** > Factory Walls Settings | **Mezzanine** > Mezzanine Settings |
| **createFactoryWalls [SimTalk]** | **createMezzanine [SimTalk]** |
| **_Factory Walls Attributes** | **_Mezzanine Attributes** |
| **Fence** > Fence Settings | **restoreCutouts [SimTalk]** |

| | |
|---|---|
| createFence [SimTalk] | _3D.getGraphic [SimTalk] |
| _Factory Walls Attributes | HasCutouts [SimTalk] |

## WallThickness [SimTalk]

Sets the thickness of the walls of the graphic designated by <Path>.

### Remarks

The attribute is available for graphics of type **Box** and **FactoryWalls**, compare *InternalGraphicType*.

You cannot assign the attribute when the graphic is an automatically generated graphic group or when the graphic is located in an automatically generated graphic group.

When you assign the attribute, *Plant Simulation* deactivates graphic inheritance.

### Type

Attribute

### Syntax

```
<Path>.WallThickness:length
```

### Assignment Value

You can assign a value of data type *length*.

### Example

```
.Models.MyModel._3D.getGraphic("deco", [3]).WallThickness := 0.02
```

### SimTalk

**InternalGraphicType [SimTalk]**

**createFactoryWalls [SimTalk]**

**createBox [SimTalk]**

**_3D.getGraphic [SimTalk]**

### See also

**InternalGraphicType [SimTalk]**

**Factory Walls**